

DevOps Interview Complete Q&A Reference

Tailored for Sanket Chopade

GII Global Infotech India Pvt. Ltd. — Fresher to 3 Years Role

Linux & OS • AWS • Docker • Kubernetes • CI/CD

Terraform • Monitoring • Networking • DevSecOps • Behavioural

Stack: AWS EKS • Terraform • Jenkins • GitHub Actions • Helm • Datadog • Prometheus

Project: ApnaKart — 10-Microservice Production Platform

■ 1. Linux & Operating System

■ BASIC

Q: What is Linux?

Linux is an open-source, Unix-like operating system kernel created by Linus Torvalds in 1991. It forms the foundation of many OS distributions (Ubuntu, CentOS, Amazon Linux). In DevOps, almost every server, container, and cloud instance runs Linux, making it the primary operating environment for all infrastructure work.

★ *Mention in interview: At Hisan Labs and on ApnaKart, all EKS worker nodes and EC2 instances ran Ubuntu/Amazon Linux.*

Q: Difference between Linux and Windows?

Linux is open-source, highly customizable, runs via CLI primarily, uses file-permission model (rwx), and is preferred for servers and containers. Windows is proprietary, GUI-focused, and dominant on desktops. In cloud/DevOps, Linux is the default choice because it is lightweight, stable, free, and has superior scripting/automation support via Bash.

Q: What is a process?

A process is a running instance of a program with its own memory space, PID (process ID), and system resources. Every command you run — like nginx, java, or a Jenkins agent — becomes a process. Use 'ps aux' or 'top' to view running processes.

Q: Difference between process and thread?

A process is an independent program instance with its own memory. A thread is a unit of execution within a process that shares the same memory space with other threads in the same process. Threads are lighter and faster to create. Java Spring Boot services (like in ApnaKart) are multi-threaded — each HTTP request is handled by a thread within the JVM process.

Q: How do you check running processes?

'ps aux' — lists all running processes with PID, CPU, memory usage. 'top' / 'htop' — interactive real-time view. 'pgrep nginx' — find PID of specific process. 'pidof java' — PID of Java process. In Kubernetes, use 'kubectl exec -it -- ps aux' to check processes inside a container.

Q: What does chmod 777 mean?

chmod 777 grants read (4), write (2), and execute (1) permissions to owner, group, and others — total permission for everyone. In DevOps, 777 is a security anti-pattern. We use minimal permissions: 600 for SSH private keys, 644 for config files, 755 for scripts. I applied least-privilege as part of our DevSecOps practices in ApnaKart.

Q: Difference between root user and normal user?

Root is the superuser with UID 0 and unrestricted system access. Normal users have limited permissions to their own home directory and designated resources. In production, we never run applications as root — containers in ApnaKart used non-root users specified via USER directive in Dockerfiles to reduce attack surface.

Q: What is SSH?

SSH (Secure Shell) is a cryptographic network protocol for secure remote access to servers over port 22. It uses public/private key authentication. In our workflow: EC2 instances were accessed via SSH key pairs (PEM files), Jenkins agents connected to EKS nodes via SSH, and all remote automation used SSH-based connections.

Q: How do you check disk space?

'df -h' — shows disk usage of all mounted filesystems in human-readable format. 'du -sh /path' — shows size of a specific directory. In production, high disk usage triggered Datadog/CloudWatch alerts. Jenkins build artifacts and Docker layer caches on build nodes required periodic cleanup managed by our Bash scripts.

Q: What is the purpose of /etc/passwd?

/etc/passwd stores user account information: username, UID, GID, home directory, and default shell — one line per user. Passwords are NOT stored here (they're in /etc/shadow, hashed). In Docker containers, this file determines which user the process runs as, relevant when enforcing non-root execution.

■ INTERMEDIATE

Q: Difference between soft link and hard link?

A hard link is a direct reference to the inode — deleting the original file doesn't affect the hard link. A soft (symbolic) link is a pointer to the file path — if the original is deleted, the symlink breaks. In DevOps, symlinks are used to manage versioned configs (e.g., /etc/nginx/sites-enabled/ → sites-available/).

Q: Explain file permissions in Linux.

Linux permissions have three classes — owner (u), group (g), others (o) — and three types: read (r=4), write (w=2), execute (x=1). 'ls -l' shows permissions like '-rwxr-xr--'. chmod changes permissions: 'chmod 755 script.sh' means owner can do all, group and others can read/execute. Critical in CI/CD: Jenkins build scripts require execute permission.

Q: What happens during Linux boot process?

1) BIOS/UEFI runs POST. 2) Bootloader (GRUB) loads the kernel. 3) Kernel initializes hardware and mounts root filesystem. 4) systemd (PID 1) starts and runs all system services. 5) Login prompt appears. In EC2, the boot sequence is abstracted but systemd still manages service startup — relevant when configuring services to auto-start after reboot.

Q: How do you monitor CPU and memory usage?

'top' or 'htop' — real-time CPU/memory. 'vmstat 1' — CPU, memory, I/O per second. 'free -h' — memory overview. 'iostat' — disk I/O. In our ApnaKart setup, Prometheus node_exporter scraped these metrics and Grafana dashboards visualized them. Datadog APM provided per-container CPU/memory with alerting thresholds.

Q: Difference between grep, awk, and sed?

grep searches for patterns in text ('grep ERROR app.log'). awk processes structured text by field ('awk "{print \$1, \$5}" access.log'). sed performs stream editing/substitution ('sed -i "s/localhost/prod-db/g" config.yml'). In CI/CD pipelines, I used these to parse build logs, extract test results, and do config templating in Bash scripts.

Q: What is systemd?

systemd is the init system and service manager in modern Linux (PID 1). It manages service start/stop/restart, dependency ordering, logging (journald), and socket activation. Key commands: 'systemctl start/stop/status nginx', 'systemctl enable nginx' (start on boot), 'journalctl -u nginx -f' (follow logs). Jenkins was managed as a systemd service on our build server.

Q: How does cron work?

cron is a time-based job scheduler. Jobs are defined in crontab files with format: minute hour day month weekday command. Example: '0 2 * * * /opt/backup.sh' runs at 2 AM daily. 'crontab -e' edits current user's crontab. In our setup, cron jobs ran nightly cleanup of old Docker images and log rotation on Jenkins build nodes.

Q: How do you troubleshoot a slow server?

Step 1: Check CPU — 'top', look for high %CPU processes. Step 2: Check memory — 'free -h', check for swap usage. Step 3: Check disk I/O — 'iostat -x 1'. Step 4: Check network — 'netstat -s', 'iftop'. Step 5: Check running processes — 'ps aux --sort=-%cpu | head'. Step 6: Check application logs. This 5-step drill was my standard approach when Datadog alerted on response time spikes in ApnaKart.

★ *Mention in interview: ApnaKart: traced latency spikes to a Spring Boot microservice with unclosed DB connections — fixed by tuning HikariCP pool settings.*

Q: What are zombie processes?

A zombie process is a process that has finished execution but still has an entry in the process table because its parent hasn't read its exit status (wait() call). They don't consume CPU but waste PIDs. Use 'ps aux | grep Z' to find them. They're cleaned when the parent process exits or calls wait(). Not directly an issue in containers (PID 1 problem — use 'tini' as init).

■ ADVANCED / SCENARIO

Q: Server disk is suddenly full. How do you investigate?

1) 'df -h' — identify which partition is full. 2) 'du -sh /* 2>/dev/null | sort -rh | head -20' — find top directories. 3) Common culprits: /var/log (log files), /tmp, Docker images ('docker system df'), Jenkins workspace. 4) Clean up: 'docker system prune -af', 'journalctl --vacuum-time=7d', remove old build artifacts. 5) Add disk monitoring alerts in Datadog at 80% threshold to catch early.

★ *Mention in interview: In our CI/CD server, Docker build cache was consuming 40GB. Added a nightly 'docker builder prune' cron job.*

Q: High CPU usage in production — what steps will you take?

1) Identify the process: 'top' or Datadog alert. 2) Check if it's expected load or abnormal spike. 3) For Java/Spring Boot: take thread dump ('kill -3 ') and heap dump. 4) Check application logs for errors or infinite loops. 5) Scale horizontally if load-related: kubectl scale deployment or trigger HPA. 6) If runaway process: isolate and restart the specific pod. 7) Post-incident: set HPA CPU thresholds and add Datadog CPU alerts.

★ *Mention in interview: In ApnaKart, the order-service spiked during load testing. HPA automatically scaled from 2 to 6 pods. We also found an N+1 query and fixed it.*

Q: Application works locally but fails on server. How will you debug?

Checklist: 1) Environment variables — missing config in prod? ('kubectl describe pod' or check ConfigMap). 2) Port/network — firewall rules, Security Groups, Ingress config. 3) Dependencies — DB, Redis, external APIs accessible from server? 4) Resource limits — OOMKilled? ('kubectl describe pod | grep -i oom'). 5) Image — correct tag deployed? 6) Logs: 'kubectl logs --previous'. 7) 'kubectl exec' into container and reproduce manually.

Q: A service fails after reboot. How do you troubleshoot?

1) Check service status: 'systemctl status '. 2) Check journal logs: 'journalctl -u -b' (since last boot). 3) Verify it's enabled for autostart: 'systemctl is-enabled '. 4) Check dependencies: does it start before its dependency? Fix with 'After=' in unit file. 5) In Kubernetes: verify pod restartPolicy is Always. 6) For EC2: check user-data script and cloud-init logs ('/var/log/cloud-init-output.log').

Q: How do you identify memory leaks in Linux?

1) Watch memory growth over time: 'watch -n 5 free -h'. 2) Track per-process memory: 'ps aux --sort=-%mem | head'. 3) Valgrind for C/C++ programs. 4) For Java: heap dump analysis with VisualVM or Eclipse MAT — look for retained objects. 5) In Kubernetes: OOMKilled pod indicates leak. Set memory limits and use Prometheus 'container_memory_working_set_bytes' metric to trend over time.

★ *Mention in interview: In ApnaKart, the inventory-service had a memory leak from cached response objects not being evicted. Fixed by adding cache TTL and monitoring with Datadog JVM metrics.*

■ 2. AWS / Cloud

■ BASIC

Q: What is cloud computing?

Cloud computing delivers on-demand IT resources (compute, storage, databases, networking) over the internet with pay-as-you-go pricing. Key models: IaaS (EC2 — you manage OS up), PaaS (Elastic Beanstalk), SaaS (Gmail). Benefits: elasticity, no upfront hardware cost, global availability, managed services. ApnaKart was 100% cloud-native on AWS.

★ *Mention in interview: Your entire ApnaKart platform is the answer — mention EKS, RDS, S3, ALB, CloudFront.*

Q: What is EC2?

EC2 (Elastic Compute Cloud) is AWS's virtual machine service. You choose instance type (CPU/RAM/network), AMI (OS image), storage, and networking. In our Terraform setup, EC2 instances were provisioned for Jenkins build servers. EKS worker nodes are also EC2 instances (managed node groups). We used t3.medium for dev and m5.large for production workloads.

Q: What is S3?

S3 (Simple Storage Service) is AWS's object storage — unlimited, durable (99.999999999%), and scalable. Used in ApnaKart for: Terraform state backend (with DynamoDB locking), Jenkins artifact storage, Docker build cache, CloudFront origin for static React frontend assets. Access controlled via IAM policies and bucket policies. Lifecycle rules managed cost by moving old artifacts to Glacier.

Q: What is IAM?

IAM (Identity and Access Management) controls who can do what in AWS. Key components: Users (humans), Groups, Roles (for services/EC2/EKS pods), Policies (JSON permission documents). In ApnaKart: EKS pods used IAM Roles for Service Accounts (IRSA) to access S3 and ECR without storing credentials. Jenkins assumed a role via EC2 instance profile. We enforced least-privilege throughout.

Q: What is VPC?

VPC (Virtual Private Cloud) is your isolated network within AWS. In ApnaKart, our Terraform VPC module created: 3 public subnets (ALB, NAT Gateway), 3 private subnets (EKS nodes, RDS), route tables, internet gateway, and NAT gateway. This ensured EKS worker nodes and RDS had no direct internet exposure — all traffic routed through ALB.

Q: Difference between public and private subnet?

Public subnet has a route to the Internet Gateway — resources get public IPs and can communicate directly with the internet. Private subnet has NO route to the Internet Gateway — resources use NAT Gateway for outbound internet access but are unreachable from the internet. In ApnaKart: ALB in public subnet, EKS nodes + RDS in private subnets. This is the standard production security pattern.

Q: What is Auto Scaling?

Auto Scaling automatically adjusts the number of EC2 instances or EKS pods based on demand. Two types: EC2 Auto Scaling Groups (scales EC2 for EKS node groups) and HPA in Kubernetes (scales pods). In ApnaKart, HPA scaled pods based on CPU utilization threshold of 70%. EKS Cluster Autoscaler added/removed nodes based on pending pod demand.

★ *Mention in interview: Mention: HPA in ApnaKart reduced manual intervention and handled 3x traffic spikes during testing.*

Q: What is Route 53?

Route 53 is AWS's DNS service. Used in ApnaKart to route apnakart.com to CloudFront distribution (for frontend) and api.apnakart.com to ALB (for backend). Supports routing policies: Simple, Weighted, Latency-based, Failover. Health checks can trigger automatic DNS failover. Integrated with ACM for SSL certificate validation.

Q: What is CloudWatch?

CloudWatch is AWS's native monitoring service for metrics, logs, and alarms. Collects EC2 CPU/network metrics, EKS control plane logs, ALB request counts, RDS metrics. In ApnaKart, CloudWatch Logs received container logs from EKS via Fluent Bit, and CloudWatch Alarms triggered SNS notifications on threshold breaches. Used alongside Datadog for full observability.

■ INTERMEDIATE

Q: Security Group vs NACL?

Security Groups are stateful, instance-level firewalls — return traffic is automatically allowed. NACLs (Network ACLs) are stateless, subnet-level firewalls — you must explicitly allow inbound AND outbound. Security Groups evaluate all rules; NACLs process rules in order (lowest number wins). In ApnaKart: SG on EKS nodes allowed only ALB traffic on port 8080; NACL on private subnets blocked all direct internet inbound.

★ *Mention in interview: This is a HIGH-frequency interview question. Know stateful vs stateless cold.*

Q: Explain AWS Load Balancer types.

ALB (Application Load Balancer): Layer 7, HTTP/HTTPS, path/host-based routing — used in ApnaKart to route /api/* to backend microservices and /* to CloudFront. NLB (Network Load Balancer): Layer 4, TCP/UDP, ultra-low latency, static IP. CLB (Classic): legacy, avoid. In Kubernetes, ALB Ingress Controller creates ALB from Ingress resource annotations automatically.

Q: How does IAM Role differ from IAM User?

IAM User is a permanent identity with long-term credentials (access key + secret). IAM Role is a temporary identity assumed by AWS services, EC2 instances, Lambda, or EKS pods — no long-term credentials. Best practice: never use IAM Users in applications. Use Roles. In ApnaKart: EKS pods used IRSA (IAM Roles for Service Accounts) — pods assumed roles via OIDC provider, no secrets stored in code.

Q: Explain EKS architecture.

EKS (Elastic Kubernetes Service) has two planes: Control Plane (AWS-managed — API server, etcd, scheduler, controller-manager) and Data Plane (your EC2 nodes or Fargate). Worker nodes join the cluster via the aws-auth ConfigMap. In ApnaKart: Terraform provisioned EKS cluster + managed node group (m5.large, 3-6 nodes with Cluster Autoscaler). Pods ran in private subnets, ALB in public subnets via Ingress.

★ *Mention in interview: Draw this architecture on a whiteboard during interview — it shows deep practical understanding.*

Q: Difference between ECS and EKS?

ECS (Elastic Container Service) is AWS-proprietary container orchestration — simpler, tighter AWS integration, less portable. EKS is managed Kubernetes — industry standard, portable across clouds, steeper learning curve but gives full Kubernetes API. ECS uses Task Definitions; EKS uses Pod specs. I chose EKS for ApnaKart because Kubernetes is the industry standard, supports Helm charts, and has a rich ecosystem.

Q: Explain CloudFront.

CloudFront is AWS's CDN (Content Delivery Network) with 450+ edge locations globally. It caches content close to users, reducing latency. In ApnaKart: React frontend was built and deployed to S3, CloudFront served it with HTTPS (ACM certificate), custom domain via Route 53. Cache invalidation triggered by Jenkins after each frontend deployment. Also added WAF to CloudFront for DDoS protection.

■ ADVANCED / SCENARIO

Q: Website is down on AWS. How will you troubleshoot?

Layer-by-layer approach: 1) DNS: nslookup/dig — does domain resolve? (Route 53). 2) CDN: CloudFront error? Check distributions status. 3) ALB: Target group health checks — are targets healthy? 4) EC2/EKS nodes: running? Security Group allowing traffic? 5) Pod/Container: 'kubectl get pods' — any CrashLoopBackOff or Pending? 6) Application logs: 'kubectl logs'. 7) Database: RDS accessible from pods? Check SG rules. This exact checklist was our runbook at Hisan Labs.

★ *Mention in interview: This is likely to be asked as a scenario question. Practice this answer until it flows naturally.*

Q: EC2 is reachable internally but not publicly. Why?

Most likely causes: 1) Security Group — inbound rule for port 80/443 missing for 0.0.0.0/0. 2) EC2 in private subnet — no public IP, no IGW route. 3) NACL — inbound or outbound rule blocking. 4) Application not listening on 0.0.0.0 (listening only on 127.0.0.1). 5) Instance has no public/Elastic IP. Diagnostic: check SG inbound rules, subnet route table for IGW, NACL rules, and 'netstat -tlnp' inside instance.

Q: How would you secure an AWS production environment?

1) IAM: least-privilege, IRSA for pods, no root account usage, MFA enabled. 2) Network: private subnets for compute/DB, SG with minimal ports, VPC Flow Logs. 3) Encryption: S3 SSE, RDS encryption at rest, TLS in transit. 4) Secrets: AWS Secrets Manager (not hardcoded). 5) Scanning: Trivy on all images in CI/CD, GuardDuty for threat detection. 6) Audit: CloudTrail for all API calls. 7) WAF on CloudFront/ALB. In ApnaKart, steps 1-5 were implemented.

★ *Mention in interview: Mention DevSecOps practices — Trivy and SonarQube in your pipeline.*

Q: How would you reduce AWS costs?

1) Right-size instances — CloudWatch metrics to identify over-provisioned EC2. 2) Reserved Instances or Savings Plans for predictable workloads (up to 72% savings). 3) Spot Instances for non-critical/batch workloads. 4) S3 Lifecycle Policies — move old artifacts to Glacier. 5) Remove unused resources — unattached EBS volumes, idle ELBs. 6) EKS: pack pods efficiently, use node autoscaler. 7) CloudFront reduces data transfer costs. 8) Use Cost Explorer and set budgets/alerts.

■ 3. Docker

■ BASIC

Q: Difference between VM and container?

VMs virtualize hardware — each VM has a full OS, hypervisor overhead, boots in minutes, uses GBs of RAM. Containers share the host OS kernel — lightweight (MBs), start in seconds, isolated via namespaces/cgroups. Docker containers are the building blocks of our microservices. In ApnaKart, 10 Spring Boot services ran as separate containers on EKS nodes — no OS overhead per service.

Q: What is Docker image?

A Docker image is a read-only, layered template used to create containers. Built from a Dockerfile. Each instruction (FROM, RUN, COPY) creates a layer — layers are cached and reused. In ApnaKart: dual registry strategy — DockerHub for dev/test images, AWS ECR for uat/prod. Images were tagged with git commit SHA for traceability. Trivy scanned all images in the Jenkins pipeline before push.

Q: Explain Docker lifecycle.

Image pulled → Container created (docker run) → Container running (process executing) → Container stopped (process exits or docker stop) → Container removed (docker rm). In CI/CD: Jenkins builds image (docker build) → pushes to ECR (docker push) → Kubernetes pulls and runs containers. Old images removed from ECR by lifecycle policy after 30 days.

■ INTERMEDIATE

Q: CMD vs ENTRYPOINT?

ENTRYPOINT defines the executable that always runs — it cannot be overridden (only appended to). CMD provides default arguments that CAN be overridden at runtime. Best practice: ENTRYPOINT for the binary ('java'), CMD for default args ('-jar app.jar'). So 'docker run image -Xmx512m' replaces CMD but keeps ENTRYPOINT. In ApnaKart Spring Boot images: ENTRYPOINT ["java"] CMD ["-jar", "/app/service.jar"].

Q: How do you optimize Docker image size?

1) Multi-stage builds — build in one stage, copy only the JAR to a slim runtime image. This reduced ApnaKart images from 600MB to ~180MB. 2) Use alpine or distroless base images. 3) Combine RUN commands to reduce layers. 4) Use .dockerignore to exclude node_modules, .git, target/. 5) Remove package manager cache: 'apt-get clean'. 6) Don't install debug tools in production images.

★ Mention in interview: Flight Reservation project: 60% image size reduction via multi-stage builds — mention this metric.

Q: Explain multi-stage builds.

Multi-stage builds use multiple FROM statements in one Dockerfile. Each stage has a name. You copy artifacts between stages, keeping the final image lean. Example for Spring Boot: Stage 1 (builder): FROM maven:3.9-openjdk-17 → runs mvn package. Stage 2 (runtime): FROM eclipse-temurin:17-jre-alpine → COPY --from=builder /target/app.jar. Final image has only the JRE + JAR, not Maven, source code, or build tools.

Q: How do you persist data in Docker?

1) Volumes: docker-managed, persist beyond container lifecycle — 'docker volume create'. 2) Bind mounts: host directory mounted into container — for dev/config files. 3) tmpfs: in-memory, non-persistent. In Kubernetes (ApnaKart): StatefulSets with PersistentVolumeClaims (EBS) for stateful services. Stateless Spring Boot microservices had no persistent storage — externalised state to RDS.

■ ADVANCED / SCENARIO

Q: Container is running but application is inaccessible. How do you debug?

1) Check if app is listening: 'docker exec -it netstat -tlnp'. 2) Check port mapping: 'docker ps' — is -p 8080:8080 present? 3) Check application logs: 'docker logs --tail 50'. 4) Test from inside container: 'curl localhost:8080/health'. 5) Check if health check is failing. 6) Verify firewall/Security Group allows the port. In Kubernetes: same steps via 'kubectl exec' and 'kubectl logs'.

Q: How do you secure Docker containers?

1) Non-root user in Dockerfile (USER 1001). 2) Read-only filesystem where possible (--read-only). 3) No privileged containers. 4) Scan images with Trivy in CI/CD — we achieved zero critical CVEs in ApnaKart prod. 5) Use specific image tags, never 'latest'. 6) Secrets via environment variables from Kubernetes Secrets/Secrets Manager, not hardcoded. 7) Network policies to restrict inter-container communication.

■ 4. Kubernetes

■ BASIC

Q: What is Kubernetes?

Kubernetes (K8s) is an open-source container orchestration platform. It automates deployment, scaling, self-healing, and load balancing of containerised applications. In ApnaKart, Kubernetes on AWS EKS managed all 10 Spring Boot microservices — handling pod restarts on failure, distributing traffic, auto-scaling on CPU thresholds, and rolling out deployments with zero downtime.

Q: Difference between Pod and Deployment?

A Pod is the smallest deployable unit — one or more containers sharing network and storage. It's ephemeral. A Deployment manages a set of identical Pods via a ReplicaSet — it maintains desired replica count, handles rolling updates, and rollback. You always use Deployments in production, not bare Pods. In ApnaKart: each of the 10 microservices had its own Deployment with min 2 replicas.

Q: Types of Kubernetes services?

ClusterIP (default): internal-only, accessible within cluster. NodePort: exposes service on a static port on each node — used for dev/testing. LoadBalancer: provisions cloud load balancer (ALB on AWS) — used for external traffic. ExternalName: maps service to external DNS. In ApnaKart: microservices used ClusterIP for internal communication; Ingress (ALB Ingress Controller) was the single external entry point.

Q: What is Ingress?

Ingress is a Kubernetes resource that manages external HTTP/HTTPS access to services using rules (path-based or host-based routing). It requires an Ingress Controller. In ApnaKart: AWS ALB Ingress Controller was installed via Helm. Ingress rules routed `/api/orders/*` → order-service, `/api/products/*` → product-service, etc. TLS termination happened at the ALB with ACM certificate.

Q: What is ConfigMap?

ConfigMap stores non-sensitive configuration data as key-value pairs — decoupling config from container images. Used in ApnaKart for: application.properties overrides, environment-specific DB URLs, feature flags. Mounted as volume or environment variables into pods. Changing ConfigMap + rolling restart avoids rebuilding images for config changes.

Q: What is Secret?

Kubernetes Secret stores sensitive data (passwords, tokens, keys) base64-encoded. Types: Opaque (generic), kubernetes.io/tls (TLS certs), kubernetes.io/dockerconfigjson (registry credentials). In ApnaKart: DB passwords and ECR pull credentials stored as Secrets, referenced in Deployment env vars. Note: base64 is encoding, not encryption — for production-grade security, use AWS Secrets Manager with External Secrets Operator.

★ *Mention in interview: Be honest that base64 K8s Secrets is not production-grade. Show awareness of ESO/Vault.*

■ INTERMEDIATE

Q: Explain Kubernetes architecture.

Control Plane: API Server (entry point for all kubectl commands), etcd (distributed key-value store — cluster state), Scheduler (assigns pods to nodes), Controller Manager (ensures desired state). Worker Nodes: kubelet (communicates with API server, runs pods), kube-proxy (network rules), container runtime (containerd). In EKS: control plane is AWS-managed (you pay per cluster); worker nodes are your EC2 instances.

Q: What is HPA?

Horizontal Pod Autoscaler automatically scales pod replicas based on CPU utilisation, memory, or custom metrics. Requires metrics-server. In ApnaKart: 'kubectl autoscale deployment order-service --cpu-percent=70 --min=2 --max=10'. During load testing, when CPU crossed 70%, HPA scaled order-service from 2 to 6 replicas within 90 seconds. Scaled back down when load dropped, saving EC2 costs.

★ *Mention in interview: Know the difference from VPA (Vertical Pod Autoscaler — changes resource requests).*

Q: What are liveness and readiness probes?

Liveness probe: checks if container is alive. If it fails, kubelet restarts the container. Readiness probe: checks if container is ready to serve traffic. If it fails, pod is removed from Service endpoints (no traffic sent). In ApnaKart, all microservices had both: liveness on /actuator/health/liveness, readiness on /actuator/health/readiness. This prevented traffic from reaching pods still warming up.

★ *Mention in interview: Without readiness probes, you get 502 errors during deployments. This is a critical production pattern.*

Q: What is RBAC?

RBAC (Role-Based Access Control) restricts Kubernetes API access. Components: Role (namespace-scoped permissions) or ClusterRole (cluster-wide), RoleBinding (links role to user/service account). In ApnaKart: each microservice had its own ServiceAccount with minimal RBAC — only permissions needed (e.g., order-service SA could only read ConfigMaps in its namespace). Jenkins SA had deploy permissions only.

Q: What is Helm?

Helm is the package manager for Kubernetes — like apt/yum for Linux. Charts are templated Kubernetes manifests. 'helm install', 'helm upgrade', 'helm rollback'. In ApnaKart: installed AWS ALB Ingress Controller, Cluster Autoscaler, Datadog DaemonSet, Prometheus-Grafana stack via Helm charts. Custom ApnaKart microservice charts had values files per environment (dev/test/uat/prod).

★ *Mention in interview: Know: helm upgrade --install -f values.yaml --namespace*

■ ADVANCED / SCENARIO

Q: Pod is in CrashLoopBackOff. How do you debug?

Step 1: 'kubectl describe pod ' — look at Events section for OOMKilled, image pull errors, config issues. Step 2: 'kubectl logs --previous' — logs from last crashed container. Step 3: Check if image exists in ECR/DockerHub. Step 4: Check resource limits — OOMKilled means memory limit too low. Step 5: Verify all environment variables and Secrets are mounted. Step 6: 'kubectl exec -it -- /bin/sh' to debug interactively (if it starts). In ApnaKart: most CrashLoopBackOff was missing env vars or wrong DB connection string.

★ *Mention in interview: This is THE most common K8s scenario question. Answer must be methodical and from experience.*

Q: How do you perform zero-downtime deployments?

1) Rolling Update (default): Kubernetes replaces pods gradually — 'maxSurge: 1, maxUnavailable: 0' ensures at least N healthy pods always running. 2) Blue-Green: deploy new version alongside old, switch traffic via Ingress. 3) Canary: route small % of traffic to new version using weighted Ingress rules. In ApnaKart: rolling updates for regular releases (98% success rate), blue-green for major version changes. Readiness probes prevented premature traffic routing.

Q: Node becomes NotReady. What will you check?

1) 'kubectl describe node ' — look at Conditions and Events. 2) 'kubectl get pods -A --field-selector spec.nodeName=' — see affected pods. 3) SSH to node: check kubelet status ('systemctl status kubelet'), disk space ('df -h'), memory ('free -h'). 4) Check AWS: is EC2 instance running? Is it passing health checks? 5) If unrecoverable: cordon node ('kubectl cordon'), drain it ('kubectl drain'), terminate and let Cluster Autoscaler replace. In EKS managed node groups, AWS often auto-replaces failed nodes.

■ 5. CI/CD — Jenkins + GitHub Actions

■ BASIC

Q: What is CI/CD?

CI (Continuous Integration): developers frequently merge code to main branch; automated build, test, scan runs on every commit. CD (Continuous Delivery/Deployment): automatically deliver tested code to staging or production. In ApnaKart: every git push triggered Jenkins — Maven build → SonarQube analysis → Trivy scan → Docker build → ECR push → Kustomize manifest update → kubectl apply to EKS. Full pipeline: 10 minutes.

★ *Mention in interview: Reduced deployment time from 45 minutes to 10 minutes (77% reduction) — always quote this metric.*

Q: What is Jenkins?

Jenkins is an open-source automation server for building CI/CD pipelines. Extensible via 1800+ plugins. In ApnaKart: Jenkins ran on EC2, connected to GitHub via webhooks. Declarative Jenkinsfile in each repo defined: build stages, SonarQube gate, Trivy scan, Docker build/push, and Kustomize deployment. Credentials (ECR token, kubeconfig) stored in Jenkins credentials store.

Q: What happens after git push?

1) GitHub receives push. 2) GitHub webhook fires POST request to Jenkins. 3) Jenkins triggers the pipeline for that branch. 4) Pipeline: checkout code → Maven build → unit tests → SonarQube analysis → quality gate check → Trivy image scan → Docker build → push to ECR → update Kustomize image tag → kubectl apply → Kubernetes rolls out new pods → readiness probes verify health. 5) Jenkins marks build SUCCESS/FAILURE. Notifications sent.

★ *Mention in interview: In ApnaKart, branch-to-namespace strategy: feature/* → dev, release/* → uat, main → prod.*

■ INTERMEDIATE

Q: Declarative vs Scripted pipeline?

Declarative: structured, uses 'pipeline { agent { stages { } } }' syntax — easier to read, validated by Jenkins, used for most cases. Scripted: uses Groovy 'node { stage { } }' syntax — more flexible but harder to read/maintain. In ApnaKart, we used Declarative pipelines exclusively. Declarative is now the Jenkins standard.

Q: How do you secure Jenkins credentials?

Jenkins Credentials Manager stores secrets encrypted. Types: Username/Password, Secret text, Secret file, SSH private key, Certificate. Referenced in Jenkinsfile as: 'withCredentials([string(credentialsId: "ecr-token", variable: "ECR_TOKEN")])'. Secrets never appear in build logs. In ApnaKart: ECR credentials, kubeconfig, SonarQube token, DockerHub credentials all managed this way — never hardcoded.

Q: How do you integrate SonarQube?

1) Install SonarQube Scanner plugin in Jenkins. 2) Configure SonarQube server URL + token in Jenkins Global Config. 3) In Jenkinsfile: wrap with 'withSonarQubeEnv("sonarqube") { sh "mvn sonar:sonar" }'. 4) Add Quality Gate check: 'waitForQualityGate()'. If gate fails (coverage < 80%, critical bugs found), pipeline aborts. In ApnaKart: SonarQube ran on every build — consistently maintained zero critical vulnerabilities.

★ *Mention in interview: Mention: SonarQube quality gate blocked 3 code pushes in ApnaKart during development.*

■ ADVANCED / SCENARIO

Q: Jenkins build fails. How do you debug?

1) Check Console Output — exact error line. 2) Common failures: compilation error (Maven), test failure, SonarQube gate, Docker daemon not running, ECR auth expired, kubectl context wrong. 3) Re-run with more verbose logging. 4) SSH to Jenkins node: check disk space, Java version, Docker daemon. 5) Check GitHub webhook delivery in repo settings — did webhook fire? 6) Check Jenkins agent connectivity. In our setup, most failures were ECR token expiry (fix: IAM role refresh) or disk full (fix: cleanup stage).

Q: How do you implement rollback in the pipeline?

1) Kubernetes rolling update supports instant rollback: 'kubectl rollout undo deployment/order-service'. 2) Versioned Docker images by git commit SHA enable targeted rollback: 'kubectl set image deployment/order-service order-service=:abc123'. 3) Pipeline can add a manual approval + rollback stage. 4) Helm: 'helm rollback '. In ApnaKart: rollback was 30 seconds — just update image tag in Kustomize and apply.

Q: How do you reduce deployment time?

1) Parallel stages in Jenkins (test + scan simultaneously). 2) Docker layer caching (BuildKit). 3) Incremental Maven builds (only changed modules). 4) Pre-pull base images on agents. 5) Efficient Dockerfile (multi-stage, minimal layers). 6) Fast Kubernetes rollout with proper resource limits (no scheduling delays). In ApnaKart: reduced from 45 to 10 minutes — parallel stages saved 12 min, Docker caching saved 8 min, efficient Dockerfiles saved 5 min.

■ 6. Terraform / Infrastructure as Code

■ BASIC

Q: What is Terraform?

Terraform is HashiCorp's open-source Infrastructure as Code (IaC) tool that provisions and manages cloud resources using declarative HCL (HashiCorp Configuration Language). Write once, deploy to any cloud. In ApnaKart: all AWS infrastructure — VPC, EKS, RDS, IAM, ALB, S3, CloudFront, Route53 — provisioned via Terraform. Separate workspaces for dev/test/uat/prod environments.

Q: Terraform init/plan/apply?

'terraform init': downloads provider plugins and initialises backend (S3 in our case). 'terraform plan': creates an execution plan — shows what will be created, changed, destroyed. Always review before apply. 'terraform apply': executes the plan and provisions resources. 'terraform destroy': tears down all resources. In ApnaKart, 'plan' was a required step in our PR review process — infra changes reviewed like code changes.

Q: What is state file?

Terraform state (.tfstate) records the current state of provisioned infrastructure — maps your config to real resources. Must be stored remotely in a team environment. In ApnaKart: state stored in S3 bucket with versioning enabled, DynamoDB table for state locking (prevents concurrent applies). Never commit state file to Git — it contains sensitive data.

■ INTERMEDIATE

Q: What are modules?

Modules are reusable, self-contained Terraform configurations. A module encapsulates related resources with input variables and output values. In ApnaKart: separate modules for vpc, eks, rds, iam, alb — each in its own directory. Root module calls child modules with environment-specific variables. This allowed us to provision dev with 1 AZ and prod with 3 AZs using the same code.

Q: How do you manage secrets in Terraform?

1) Never hardcode secrets in .tf files. 2) Use 'sensitive = true' on output/variable to suppress in logs. 3) Pass secrets via environment variables (TF_VAR_db_password). 4) Use AWS Secrets Manager data source to read secrets at plan time. 5) Use Vault provider for advanced secrets. In ApnaKart: RDS password stored in SSM Parameter Store (SecureString), retrieved via Terraform data source — never in Git.

Q: What happens if Terraform state is corrupted?

Recovery steps: 1) Check S3 versioned backups — restore previous version. 2) 'terraform refresh' — sync state with actual infrastructure. 3) 'terraform import' — manually import existing resources back into state. 4) Last resort: manually reconstruct state file from 'terraform show'. Prevention: enable S3 versioning, DynamoDB locking, never manually edit .tfstate. In ApnaKart, S3 versioning on state bucket was mandatory from day one.

■ ADVANCED / SCENARIO

Q: Two engineers modified infrastructure simultaneously. What happens?

With DynamoDB state locking: Engineer 1's 'terraform apply' acquires lock. Engineer 2's attempt fails with 'Error acquiring the state lock'. Engineer 2 must wait. Without locking: both apply simultaneously — state file corruption, duplicate resources, or one engineer's changes overwrite the other's. Lesson: ALWAYS configure remote state with locking. In ApnaKart: S3 backend + DynamoDB lock table configured in terraform/backend.tf.

Q: Infrastructure drift occurred. How do you detect it?

Infrastructure drift is when real cloud resources differ from Terraform state (e.g., someone manually modified an SG in the console). Detection: 1) 'terraform plan' — will show changes needed to reconcile. 2) 'terraform refresh' — updates state to match actual resources. 3) AWS Config or Datadog CSPM for continuous drift detection. Prevention: no manual console changes, all changes through PR → Terraform. We enforced this policy at Hisan Labs.

■ 7. Monitoring & Observability

■ BASIC

Q: What is Prometheus?

Prometheus is an open-source monitoring system and time-series database. It scrapes metrics from targets at configurable intervals. Uses PromQL for queries. Pull-based model. In ApnaKart: Prometheus scraped metrics from all 10 Spring Boot microservices (via /actuator/prometheus endpoint), node exporter (OS metrics), kube-state-metrics (Kubernetes object metrics), and Datadog DaemonSet agents.

Q: What is Grafana?

Grafana is an open-source visualisation platform that creates dashboards from data sources like Prometheus, Loki, CloudWatch, Datadog. In ApnaKart: Grafana had 3 dashboards — cluster overview (CPU/memory per node), microservices dashboard (request rate, error rate, latency per service), and business metrics (orders/min, revenue trends). Alerts configured to fire to Slack for P1 incidents.

Q: Logs vs Metrics?

Metrics are numerical time-series data points (CPU %, request count, latency). Efficient to store, fast to query, good for alerting. Logs are timestamped text records of events — verbose, detailed, good for debugging. In observability (the three pillars): Metrics (Prometheus/Datadog) for alerting; Logs (ELK/CloudWatch Logs) for root-cause analysis; Traces (Datadog APM) for distributed request tracing across microservices.

■ INTERMEDIATE

Q: How does Prometheus scraping work?

Prometheus server reads 'scrape_configs' in prometheus.yml to know targets. It sends HTTP GET to /metrics at the scrape_interval. Targets expose metrics in Prometheus text format. In Kubernetes: ServiceMonitor CRD (from Prometheus Operator) automatically discovers services with matching labels. In ApnaKart: Spring Boot Actuator exposed /actuator/prometheus endpoint; ServiceMonitor selected services with label 'monitoring: enabled'.

Q: What is MTTR?

MTTR (Mean Time to Recovery/Repair) is the average time to restore service after an incident. Key SRE metric. In ApnaKart: baseline MTTR was 45 minutes before our observability stack. After implementing Prometheus alerts + Datadog APM + pre-built Grafana dashboards, MTTR reduced by 30% to ~32 minutes. Reduced because: faster detection (alerting vs manual discovery) and faster diagnosis (dashboards pre-built).

★ *Mention in interview: Always quantify: 30% MTTR reduction — this is a strong metric for a fresher.*

Q: Difference between Datadog and Prometheus?

Prometheus: open-source, self-hosted, pull-based, requires Grafana for visualisation, excellent for Kubernetes-native monitoring, free. Datadog: commercial SaaS, agent-based, APM + logs + metrics + tracing in one platform, excellent UI, AI-powered anomaly detection, costly. In ApnaKart: used both — Prometheus/Grafana for infrastructure metrics (free), Datadog APM for distributed tracing and business-level alerting (trial account).

■ ADVANCED / SCENARIO

Q: Service latency increased suddenly. Investigation steps?

1) Check Grafana service dashboard — when did latency spike? Which endpoint? 2) Check error rate — is it timing out or returning errors? 3) Datadog APM traces — which microservice in the call chain is slow? 4) Check downstream dependencies: RDS query time, Redis cache hits, external API response time. 5) Check resource saturation: CPU/memory of the slow service's pods. 6) Check recent deployments — did a code change correlate? In ApnaKart: traced a latency spike to an unindexed DB column added in a new feature release.

★ *Mention in interview: USE: 'kubectl top pods -n production' during investigation.*

Q: How do you monitor Kubernetes clusters?

Three layers: 1) Infrastructure: node CPU/memory via Prometheus node_exporter + Grafana. 2) Control plane: kube-state-metrics for pod/deployment/node object state, API server metrics. 3) Application: Spring Boot Actuator metrics scraped by Prometheus, APM traces by Datadog agent DaemonSet. Cluster-level: kubelet metrics, etcd metrics. Alerts: PodCrashLooping, NodeNotReady, HighCPUUsage, PVCNearFull. In ApnaKart all three layers were monitored.

■ 8. Networking

■ BASIC

Q: What is DNS?

DNS (Domain Name System) translates human-readable domain names (apnakart.com) to IP addresses. Process: browser checks local cache → OS resolver → Recursive resolver → Root nameserver → TLD nameserver (.com) → Authoritative nameserver (Route 53) → returns IP. TTL controls caching duration. In production: low TTL (60s) for services that may change IP; high TTL (3600s) for stable endpoints. Route 53 was our authoritative DNS for ApnaKart.

Q: What is HTTP vs HTTPS?

HTTP is plaintext web protocol on port 80 — unencrypted, insecure. HTTPS is HTTP over TLS/SSL on port 443 — encrypted, authenticates server via certificate. In ApnaKart: all traffic HTTPS only. CloudFront had ACM certificate for frontend; ALB had ACM certificate for API. HTTP traffic auto-redirected to HTTPS via ALB listener rules. Enforced HSTS headers for strict transport security.

Q: TCP vs UDP?

TCP is connection-oriented — establishes 3-way handshake (SYN, SYN-ACK, ACK), guarantees delivery and ordering, slower. UDP is connectionless — no handshake, no guarantee, faster. TCP: HTTP/HTTPS, SSH, databases, Kubernetes API server. UDP: DNS queries, video streaming, NTP. In Kubernetes: most application traffic is TCP; CoreDNS uses UDP (port 53) for DNS resolution within the cluster.

■ INTERMEDIATE

Q: Explain OSI model.

7 layers: Physical (1, cables), Data Link (2, MAC/switches), Network (3, IP/routing), Transport (4, TCP/UDP), Session (5, connection management), Presentation (6, encryption/encoding), Application (7, HTTP/DNS/SSH). DevOps relevance: troubleshooting network issues — Security Groups operate at L3/L4; ALB at L7; NLB at L4. TLS (HTTPS) operates at L5-L6. DNS at L7. 'Works at Layer 8' means user error.

Q: What is a reverse proxy?

A reverse proxy sits in front of servers, accepting client requests and forwarding to backend servers. Benefits: load balancing, SSL termination, caching, DDoS protection, hiding backend topology. In ApnaKart: ALB (reverse proxy at AWS level) and Nginx Ingress Controller (reverse proxy at Kubernetes level). CloudFront also acts as reverse proxy + CDN for the frontend. NGINX configured upstream blocks to distribute requests.

Q: What is SSL/TLS handshake?

1) Client Hello: client sends supported TLS versions + cipher suites. 2) Server Hello: server selects cipher, sends certificate. 3) Client verifies certificate against CA. 4) Key exchange: client generates pre-master secret, encrypts with server's public key. 5) Both derive session keys. 6) Encrypted communication begins. In ApnaKart: ALB terminated TLS using ACM certificate — backend pods communicated in HTTP within the VPC (private network, no need for pod-to-pod TLS in our setup).

Q: DNS resolution is failing. How do you debug?

1) 'nslookup domain.com' or 'dig domain.com' — does it resolve? 2) 'dig @8.8.8.8 domain.com' — test against Google DNS (bypass local resolver). 3) Check Route 53 hosted zone — correct records? 4) In Kubernetes: 'kubectl exec -it -- nslookup kubernetes.default' — tests CoreDNS. Check CoreDNS pods are running. 5) Check /etc/resolv.conf in pod — correct nameserver? 6) Check DNS TTL — wait for propagation after changes. Most K8s DNS issues are misconfigured CoreDNS ConfigMap.

Q: How would you troubleshoot API timeout issues?

1) Check if server is receiving requests (ALB access logs, nginx logs). 2) Check response time metrics — is timeout at network or application layer? 3) Check application thread pool — all threads occupied? (connection pool exhaustion). 4) Check downstream dependencies: DB query time, external API latency. 5) Trace with Datadog APM — identify slow span. 6) Check TCP connection state: 'netstat -an | grep TIME_WAIT'. 7) Tune timeout values: ALB idle timeout (default 60s), application keepalive settings.

■ 9. DevSecOps / Security

■ BASIC

Q: What is DevSecOps?

DevSecOps integrates security practices throughout the DevOps pipeline — 'shift left security' means catching vulnerabilities early (in code) rather than late (in production). Security becomes everyone's responsibility, not just the security team's. In ApnaKart: SonarQube (SAST — static code analysis) and Trivy (container image scanning) were mandatory pipeline stages. Build failed if critical vulnerabilities were found.

Q: What is SonarQube?

SonarQube is a static application security testing (SAST) tool that analyses source code for bugs, code smells, security vulnerabilities, and code coverage. Supports 30+ languages. Quality Gates define pass/fail thresholds. In ApnaKart: SonarQube ran in the Jenkins pipeline after Maven build. Quality gate required: 0 critical bugs, 0 critical vulnerabilities, ≥70% test coverage, maintainability rating A.

Q: What is Trivy?

Trivy is an open-source vulnerability scanner by Aqua Security. Scans Docker images, filesystems, git repositories, and Kubernetes manifests for CVEs (Common Vulnerabilities and Exposures). Fast, accurate, zero config. In ApnaKart Jenkins pipeline: `'trivy image --severity CRITICAL,HIGH --exit-code 1 .'` — pipeline fails if any CRITICAL vulnerabilities found. Result: zero critical CVEs in production across all deployments.

■ INTERMEDIATE

Q: How do you secure Kubernetes?

1) RBAC: least-privilege ServiceAccounts per workload. 2) Network Policies: restrict pod-to-pod communication (only allow necessary paths). 3) Pod Security Standards: no privileged containers, non-root users. 4) Secrets management: use External Secrets Operator with AWS Secrets Manager instead of native K8s Secrets. 5) Image scanning: Trivy in CI/CD. 6) Audit logs: enable EKS control plane logging. 7) Node security: managed node groups, IMDSv2 enforced, no SSH from internet.

Q: How do you manage secrets securely?

1) Never store secrets in code or Git. 2) Kubernetes Secrets (base64 encoded — NOT encrypted at rest by default — not production-grade). 3) AWS Secrets Manager: stores encrypted secrets, auto-rotates DB passwords, integrates with RDS. 4) External Secrets Operator (ESO): Kubernetes controller that syncs secrets from AWS Secrets Manager/SSM into K8s Secrets automatically. 5) HashiCorp Vault: enterprise-grade secrets management. In ApnaKart: used SSM Parameter Store for Terraform secrets, K8s Secrets for pod environment variables.

★ *Mention in interview: Acknowledge the gap: K8s base64 Secrets are not encrypted. Show awareness of ESO and Vault.*

■ ADVANCED / SCENARIO

Q: Vulnerability found in production image. What next?

1) Assess severity: CRITICAL/HIGH CVE in base OS or library? Is it exploitable in our context? 2) Isolate if exploitable: cordon node, restrict network policy, scale down affected deployment. 3) Patch: update base image tag in Dockerfile (e.g., eclipse-temurin:17.0.9 → 17.0.10-jre), rebuild, rescan with Trivy. 4) Deploy patched image: tested rollout. 5) Post-incident: add this CVE to Trivy ignore list only if accepted risk, document. 6) Prevention: weekly automated image rescans even without code changes.

Q: How do you implement security in CI/CD pipelines?

Layered approach: 1) Pre-commit: secrets detection (git-secrets, truffleHog). 2) Build stage: dependency scanning (OWASP Dependency-Check for Maven). 3) Code analysis: SonarQube SAST after compilation. 4) Image stage: Trivy scan before push — fail on CRITICAL. 5) Deployment: OPA/Gatekeeper policies enforce no-latest-tag, non-root user. 6) Runtime: Falco for anomaly detection. In ApnaKart: steps 2-4 implemented. Trivy gate was the most impactful — prevented 4 vulnerable images from reaching prod.

■ 10. Behavioural + Real Experience

Q: Tell me about yourself.

I'm Sanket Chopade, a DevOps Engineer with 9 months of hands-on internship experience at Hisan Labs in Pune. My core stack is AWS, Kubernetes, Terraform, and Jenkins. I built ApnaKart — a production-grade 10-microservice platform on AWS EKS — where I reduced deployment time by 77% and maintained 99.9% uptime. I'm an aeronautical engineering graduate who pivoted to DevOps because I'm driven by building automated, scalable infrastructure. I'm excited about this role at GII because of the opportunity to work on live production systems and grow in a hands-on environment.

★ *Mention in interview: Keep this to 90 seconds max. Practice it until it sounds natural, not recited.*

Q: Explain your CI/CD pipeline end-to-end.

Developer pushes code to GitHub. GitHub webhook triggers Jenkins. Jenkins pipeline runs: 1) Maven build + unit tests. 2) SonarQube static analysis — quality gate must pass. 3) Docker multi-stage build. 4) Trivy image scan — no CRITICAL CVEs allowed. 5) Push to AWS ECR (uat/prod) or DockerHub (dev/test). 6) Kustomize updates image tag in the environment-specific overlay. 7) kubectl apply to EKS namespace. 8) Readiness probes verify pods healthy before routing traffic. Total time: 10 minutes. If any step fails, pipeline stops and Slack notification fires.

★ *Mention in interview: This is your strongest answer. It shows the complete DevOps lifecycle from your hands-on experience.*

Q: Biggest challenge you faced?

The hardest challenge in ApnaKart was getting inter-microservice communication right in Kubernetes. Initially, services were failing to connect because of incorrect ClusterIP service discovery. The problem: services were hardcoded with localhost URLs instead of Kubernetes service DNS names (`http://order-service.production.svc.cluster.local:8080`). I debugged by exec-ing into containers and using `nslookup/curl` to trace the DNS resolution. Once I understood Kubernetes service discovery, I updated all Spring Boot application.properties to use DNS names and the issue was resolved. That experience made me genuinely understand K8s networking.

★ *Mention in interview: Good STAR-format story: real, technical, resolved by your own debugging.*

Q: Why DevOps?

I started as an aeronautical engineer but discovered during my internship that I love the infrastructure side of software — the automation, the systems thinking, the fact that good DevOps multiplies the productivity of every developer on the team. What hooked me was the first time I watched a Jenkins pipeline take code from a commit to a live Kubernetes deployment in under 10 minutes — something that used to take hours manually. DevOps is where engineering rigour meets real-world impact, and that's what drives me.

Q: Why should we hire you?

I bring something unusual for a fresher: real production experience. ApnaKart is not a tutorial project — it's a 10-microservice platform with proper Terraform infra, a security-gated CI/CD pipeline, and production-grade Kubernetes setup with HPA, RBAC, and Ingress. At Hisan Labs, I reduced deployment time by 77% and cut MTTR by 30%. I know the tools in your stack — Linux, Docker, Kubernetes, Jenkins, AWS, Terraform, Prometheus. I work independently, I document everything, and I'm honest about what I don't know and fast at learning it.

Q: What are your strengths and weaknesses?

Strength: I can own a problem end-to-end. From writing Terraform to debugging a CrashLoopBackOff to setting up the Grafana alert — I don't pass it to someone else. I figure it out. Weakness: I have not yet worked in a 24x7 on-call production rotation. My experience is internship-level, not enterprise incident response. I'm addressing this by studying SRE practices, setting up personal monitoring projects, and actively preparing for on-call scenarios. I know this gap exists and I'm not pretending otherwise.

★ *Mention in interview: Honest weakness + active mitigation = strongest possible answer. Don't say 'I work too hard'.*

■ Quick-Fire Cheat Sheet — Key Metrics & Commands

Metric	Value	Context
Deployment time reduction	77% (45 min → 10 min)	Jenkins CI/CD pipeline optimisation
MTTR reduction	30%	Prometheus + Datadog observability stack
System availability	99.9%	ApnaKart EKS production cluster
Critical CVEs in production	0	Trivy scanning in every pipeline run
Deployment success rate	98%	Rolling + blue-green strategies with readiness probes
Manual task elimination	70%	Bash automation scripts
Provisioning time reduction	40%	Terraform IaC vs manual AWS console
Container image size reduction	60%	Multi-stage Docker builds (Flight Reservation)
Microservices in ApnaKart	10	Spring Boot + React 18 + AWS EKS
HPA scale-out (load test)	2 → 6 pods	CPU threshold 70%, ~90 seconds
ECR push target	UAT/Prod only	DockerHub for dev/test — dual registry strategy

Essential Commands

<code>kubectl get pods -n production -o wide</code>	List pods with node assignment
<code>kubectl describe pod <pod> -n production</code>	Full pod details + events
<code>kubectl logs <pod> --previous -n production</code>	Logs from crashed container
<code>kubectl exec -it <pod> -- /bin/bash</code>	Shell into running container
<code>kubectl rollout undo deployment/<name></code>	Instant rollback
<code>kubectl top pods -n production</code>	Real-time CPU/memory per pod
<code>kubectl get events -n production --sort-by=.lastTimestamp</code>	Recent cluster events
<code>terraform plan -out=tfplan</code>	Plan with saved output
<code>terraform apply tfplan</code>	Apply saved plan safely
<code>docker system prune -af --volumes</code>	Clean all unused Docker data
<code>trivy image --severity CRITICAL,HIGH <image></code>	Scan image for critical CVEs
<code>df -h && du -sh /* 2>/dev/null sort -rh head</code>	Disk usage top directories
<code>ps aux --sort=-%cpu head -10</code>	Top CPU processes
<code>journalctl -u <service> -f</code>	Follow service logs
<code>aws eks update-kubeconfig --name apnakart --region ap-south-1</code>	Configure kubectl for EKS

■ Top 10 Questions — If You Prepare Nothing Else

1

Explain your CI/CD pipeline end-to-end

Your strongest answer. 45→10 min, SonarQube gate, Trivy, ECR, Kustomize, EKS rollout.

2

Describe ApnaKart architecture

10 Spring Boot microservices, EKS, RDS, Terraform, Helm, Datadog DaemonSet, 99.9% uptime.

3

Security Group vs NACL

Stateful vs stateless, instance vs subnet, all-rules vs ordered-rules.

4

Pod is in CrashLoopBackOff — how do you debug?

describe pod → Events; logs --previous; check image, env vars, OOM, resource limits.

5

What happens when a Jenkins build fails?

Console output → stage failure → common causes → your debugging approach.

6

How does HPA work?

Metrics server → CPU threshold → scale-out/in; ApnaKart 2→6 pods in 90s.

7

Explain Terraform workflow

init → plan (review!) → apply → destroy. State in S3 + DynamoDB lock.

8

How do you monitor applications in production?

Three pillars: metrics (Prometheus/Datadog), logs (CloudWatch/ELK), traces (Datadog APM).

9

What is IRSA and why is it better than access keys?

IAM Role for Service Account via OIDC — no long-lived credentials in pods.

10

Tell me about a production issue you solved

Inter-service DNS failure in ApnaKart — STAR format: Situation → Task → Action → Result.

Sanket Chopade · chopade.sanket@outlook.com · github.com/Sanket006 · Pune, India
Prepared for GII Global Infotech India Pvt. Ltd. — DevOps Engineer Role